Individual Contribution Reflection — Raven Ge (wege8390)

COMP4347 / COMP5347 Assignment 2 · Quiz subsystem, Review Mode, frontend integration and UI/UX ownership

1. Contribution Summary

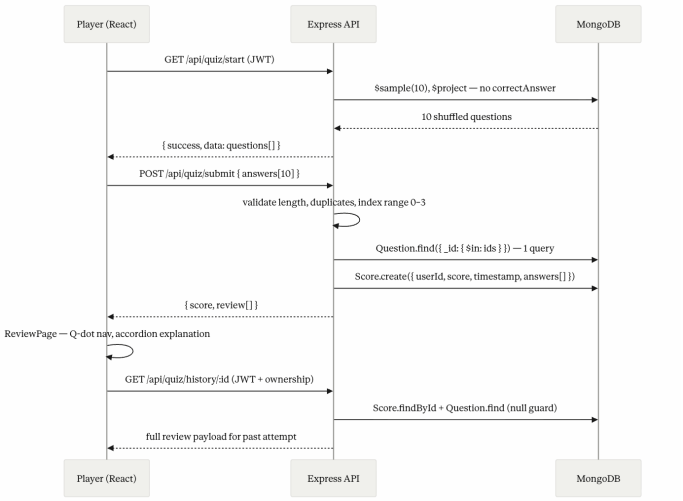
I owned the quiz subsystem across backend and frontend. On the backend (`def48eb`), I implemented `quiz.controller.js` in full: randomised question selection via MongoDB `$sample` fixed at 10, input validation (answer count, duplicate detection, index range 0–3), batch-fetch scoring replacing an N+1 loop, server-side `review[]` assembly for Review Mode, leaderboard `$lookup` join, and history retrieval with ownership validation. On the frontend (`62ca283`), I built `QuizContext.js` (useReducer state management), `Quiz.js`, `AttemptReview.js` (Review Mode page), and `History.js`. These files were lost in a subsequent merge conflict and are verifiable via `git show 62ca283`. During the integration phase I took primary ownership of the player-facing UI/UX system, including gameplay screens, Review Mode layouts, navigation components, and the Gothic Academia visual theme. AI-assisted visual assets were selectively adapted to support the Gothic-academia themed interaction design while maintaining readability and gameplay clarity.

2. Major Technical Challenge

The original `submitQuiz` called `Question.findById()` inside a loop, producing ten sequential database queries per submission. I refactored to a single `Question.find({ _id: { $in: questionIds } })` call with a hashmap lookup, reducing database round trips from 10 to 1. The same hashmap then powered Review Mode assembly at no extra cost, returning `questionText`, `options`, `correctAnswer`, and `explanation` in the submit response without additional queries.

A second challenge emerged after adding `shuffleQuestion.js` (`41f7ce8`): the submitted `selectedAnswer` index must correspond to the shuffled option order, not the seed order. I resolved this by preserving the shuffled array throughout the quiz lifecycle so that scoring and review reconstruction remained consistent with what the user actually saw during gameplay.

3. Subsystem Sequence Diagram (Mermaid)



4. Git Commit Analysis

Hash	What
`def48eb`	Rewrote `quiz.controller.js`: replaced N+1 loop with batch `Question.find()` + hashmap, added answer validation (count, duplicates, index range 0–3), assembled `review[]` server-side, added leaderboard `$lookup` and history ownership check
`62ca283`	Built `QuizContext.js` with useReducer for quiz state, `Quiz.js` for sequential question rendering, `AttemptReview.js` for Review Mode answer display, `History.js` for past attempt list. *Files lost in merge conflict — verifiable via `git show 62ca283`*
`8724146`	Established player UI system
`e22b914`	Replaced background image, fixed interaction bugs in `QuizScreens.jsx`, restructured `quizflow.css` for maintainability
`4c1730d`	Refined quiz flow transitions and animation
`268d0c6`	Rebuilt Review Mode card layout, styled History page attempt list, expanded `quizflow.css` with review-specific rules
`41f7ce8`	Added `shuffleQuestion.js` to randomise option order per attempt, updated `quiz.controller.js` scoring to use shuffled index, updated `Question.js` model, added full question seed dataset
`d17fe3c`	Fixed quiz flow state transitions, rebuilt `HistoryPage.jsx` with attempt routing, refactored login/register UI, added day/night logo assets
`657d15b`	Added `forbidAdminQuiz.middleware.js` to block admin users from taking player quizzes, added unit test, rebuilt `Leaderboard.jsx` and `HistoryPage.jsx`
`0102a29`	Created `ReviewV7Card.jsx` for per-question review display, `reviewFormatUtils.js` for answer formatting, rebuilt `ReviewPage.jsx` layout, added CSS modules for review and history screens
`d04b018`	Extracted navigation into `PlayerNavbar.jsx`, `AdminNavbar.jsx`, `PublicNavbar.jsx`, added `GameplayHudPortal.jsx` for in-game HUD, refactored `QuizScreens.jsx` routing, added gothic typography CSS
`93a9096`	CSS cleanup pass: removed redundant rules, standardised spacing and font tokens across `quizflow.css` and `quiz-play.css`
`64d01cf`	Fixed landing page routing, added `ProtectedAdminRoute.jsx` to guard admin pages, added `motion-system.css` for transition animations
`c5ea913`	Leaderboard tie-breaking: users with equal scores ranked by earliest submission timestamp; result set capped at 50 entries

5. Review Mode Design Reflection

The `review[]` array is assembled server-side during submission using the `questionMap` already loaded for scoring, avoiding an extra API call and keeping the Score model minimal. I replaced an initial flip-card design with accordion-based explanations because flip-cards imply hidden information — appropriate during gameplay, not after submission when all answers are already revealed. The empty-string guard (`if (q.explanation)`) suppresses the expand button when no explanation exists, since MongoDB stored the field as `""` rather than absent. Finally, because `shuffleQuestion.js` randomises option order per attempt, the review payload must preserve the same shuffled ordering shown during gameplay — failing to do so would produce a review screen that appears correct but explains the wrong option.